

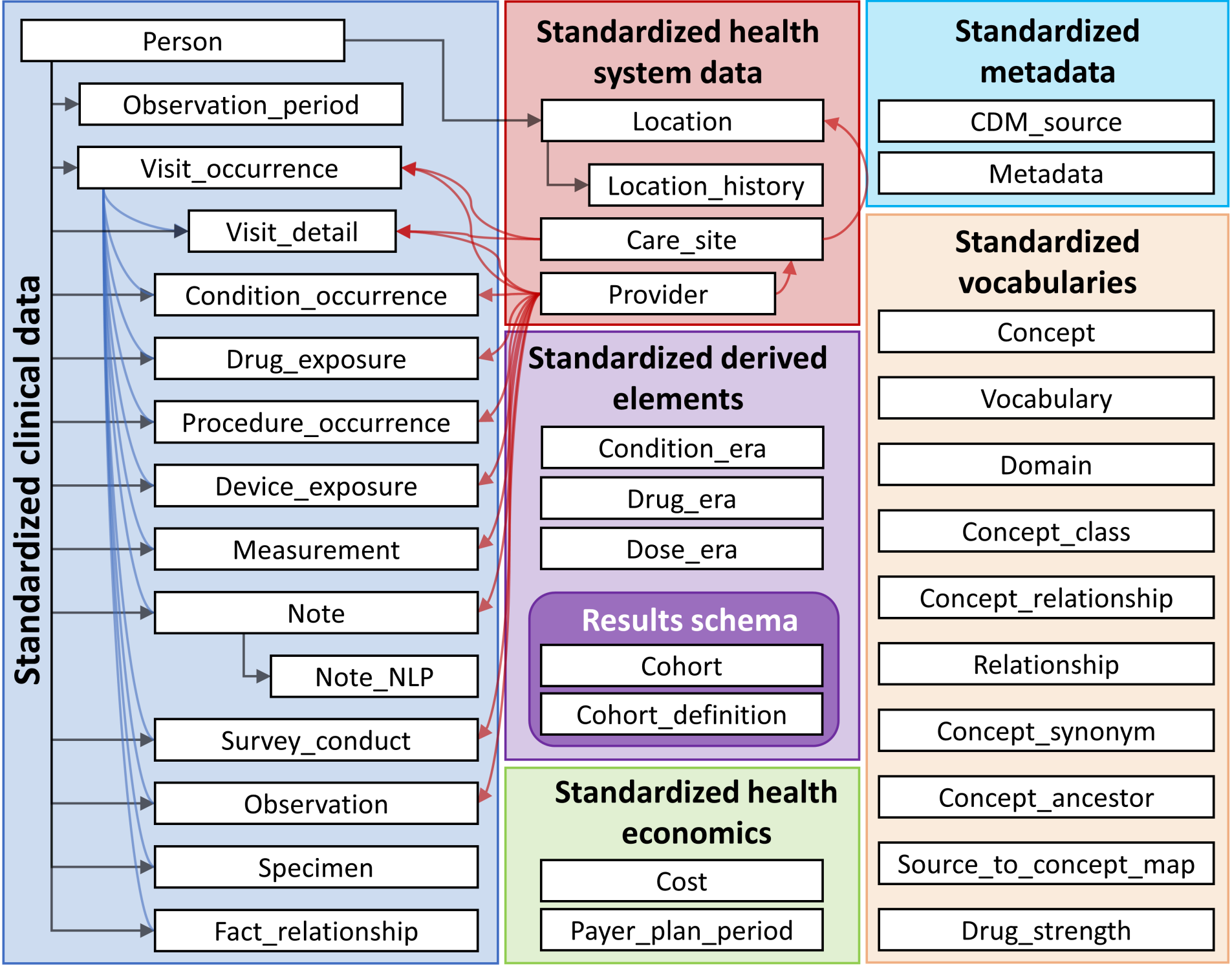
OMOP and CDMConnector

The OMOP Common Data Model

2026-06-22

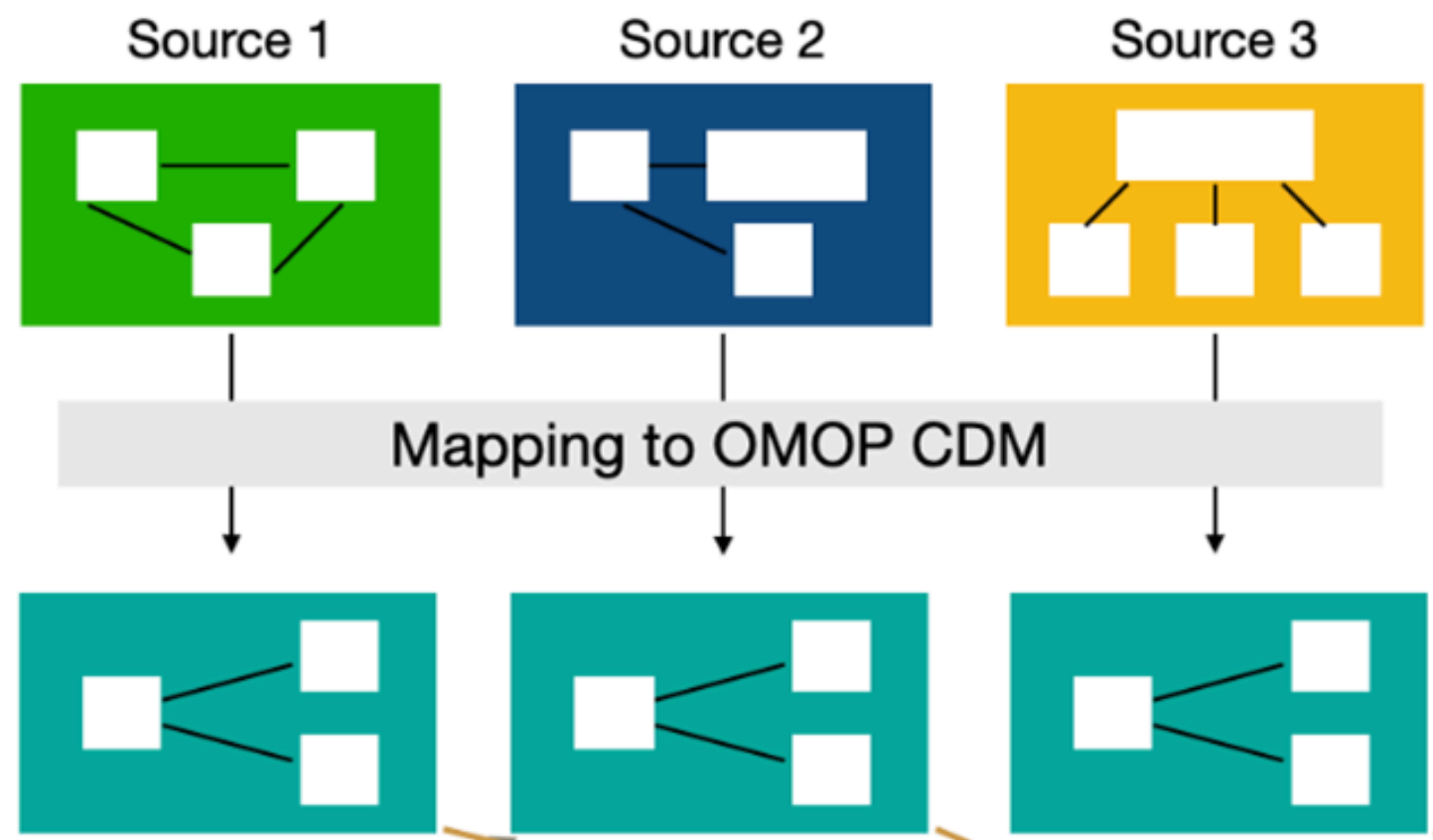
Introduction: The OMOP Common Data Model

Standardisation of the data format



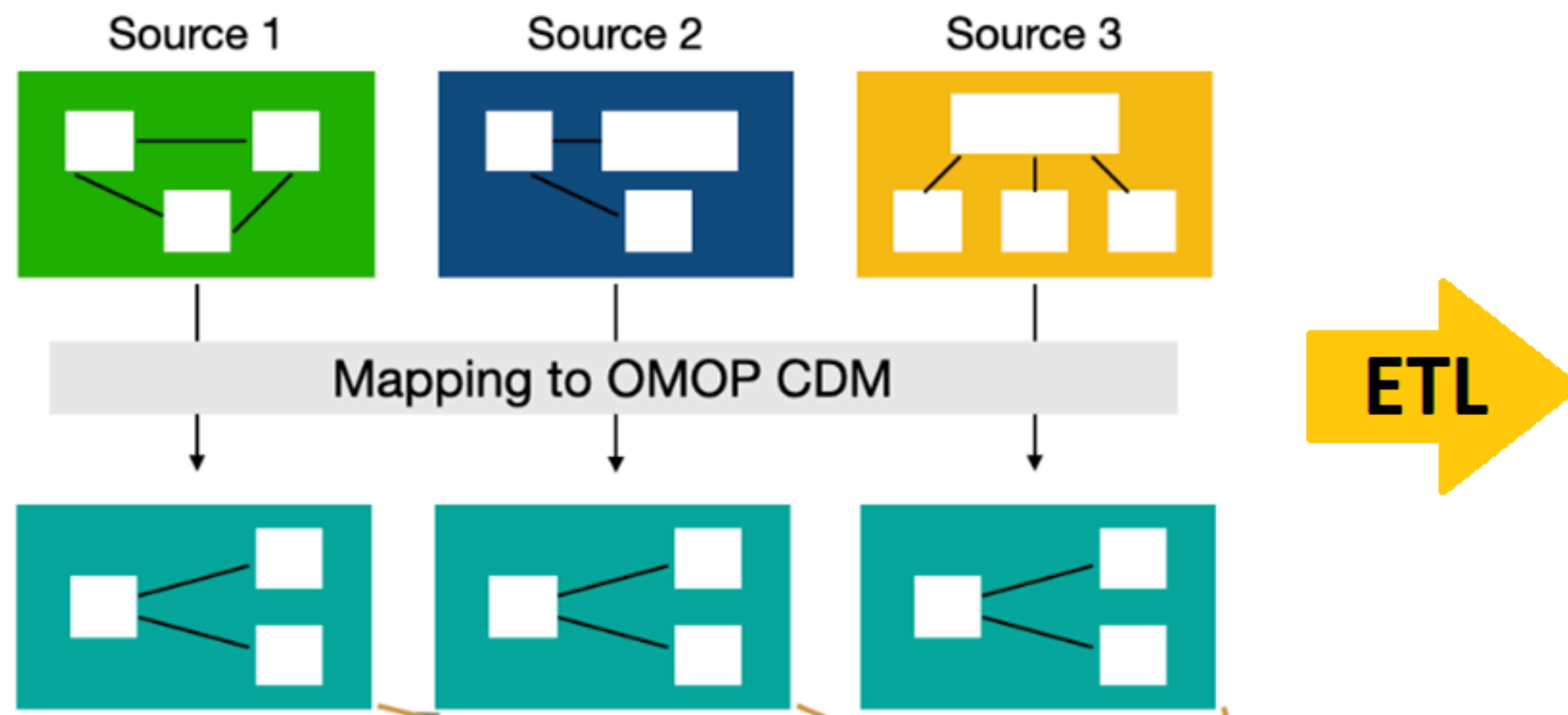
Tables and relation in the OMOP Common Data Model

Mapping a database to the OMOP CDM



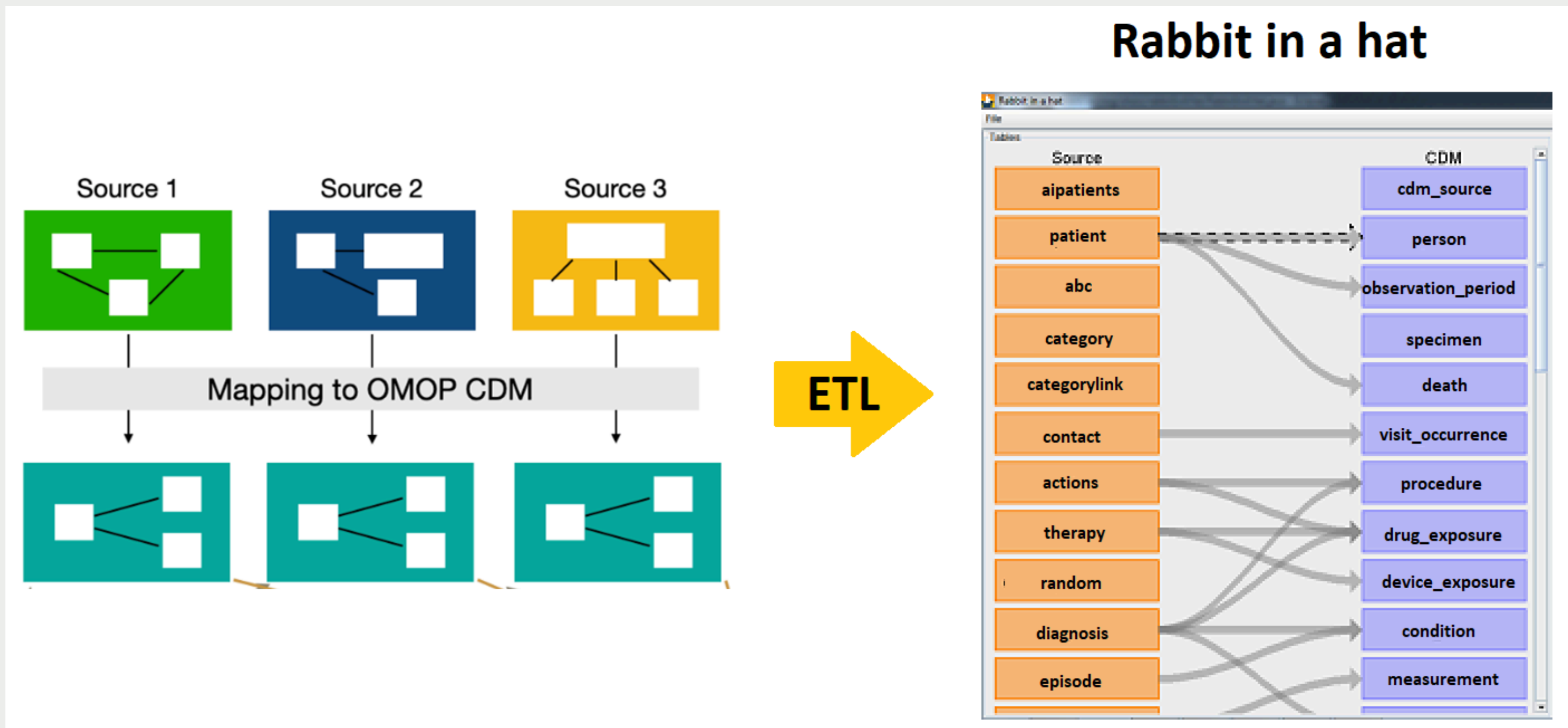
Mapping process

Mapping a database to the OMOP CDM



Mapping process

Mapping a database to the OMOP CDM



Mapping process

Standardisation of the vocabularies

From all the vocabularies OMOP CDM uses only a few as **Standard**: SNOMED for conditions, RxNorm for drugs, ...

The process to obtain an standard code from non standard one is called mapping. We can find the mapping in the concept_relationship table.

Each one of the records in clinical data tables (condition_occurrence, drug_exposure, measurement, observation, ...) will be coded by two codes:

- Source concept: particular to each database, it is the **original** code.
- Standard concept: equivalent code from the standard vocabulary.

Example of mapping

In concept relationship we can find different information such as:

```
> cdm$concept_relationship %>% select(relationship_id) %>% distinct() %>% pull()
[1] "AB-drug cjgt of" "Acc device used by" "Accepted use of" "Access of" "Active ing of" "Add monitor ind of" "Admin method of" "AMP of" "AMP restr ind of" "Answer of" "Answer of (PPI)" "Antineoplastic of"
[13] "ARP of" "Asso finding of" "Asso morph of" "Asso proc of" "Asso with finding" "ATC - RxNorm" "ATC - RxNorm pr lat" "ATC - RxNorm pr up" "ATC - RxNorm sec lat" "ATC - RxNorm sec up" "ATC - SNOMED eq" "ATC to NDFRT eq"
[25] "ATC to VA Class eq" "Available as box" "Basic dose form of" "Basis str subst of" "Biosimilar of" "Box of" "Brand name of" "CAP-Nebraska cat" "CAP-Nebraska eq" "CAP parent item of" "CAP protocol of" "CAP value of"
[37] "Category in Chapter" "Category of" "Causative agent of" "CD category of" "Chapter has Category" "Chapter to ICD0" "Characterizes" "Chem structure of" "Chem to Prep eq" "CI by" "CI chem class of" "CI MoA of"
[49] "CI physiol effect by" "CI to" "Clinical course of" "Combi prod ind of" "Comp material of" "Component of" "Conc denom unit of" "Conc denom val of" "Conc num unit of" "Conc num val of" "Concept alt_to from" "Concept alt_to to"
[61] "Concept poss_eq from" "Concept poss_eq to" "Concept replaces" "Concept same_as from" "Concept same_as to" "Concept was_a from" "Concept was_a to" "Consists of" "Constitutes" "Contained in" "Contained in panel"
[73] "Contained in version" "Contains" "Context of" "Count of act ing in" "Count of ing of" "CPT4 - LOINC eq" "CPT4 - SNOMED cat" "CPT4 - SNOMED eq" "Current treated by" "CVX - RxNorm" "Cytotox chemo RX of" "Cytotoxic chemo of"
[85] "Denom value of" "Denominator unit of" "Dev intended site of" "Device used by" "Diagnosed through" "Dir device of" "Dir morph of" "Dir proc site of" "Dir subst of" "Direct site of" "Disc indicator of" "Disp dose form of"
[97] "Disposition of" "DOI - RxNorm" "Dose form group of" "Dose form of" "Dose form unit of" "DRG - MDC cat" "DRG - MS-DRG eq" "Drug-drug inter for" "Drug class of drug" "Drug has drug class" "Due to of" "During"
[109] "End date of" "Endocrine tx of" "Energy used by" "Episodicity of" "ETC - RxNorm" "Excipient of" "FDA indication of" "FDA labeling of" "Filling of" "Finding asso with" "Finding context of" "Finding inform of"
[121] "Finding method of" "Finding site of" "Flavor of" "Focus of" "Followed by" "Follows" "Form continuity of" "Form of" "Free indicator of" "Has AB-drug cjgt" "Has AB-drug cjgt Rx" "Has accepted use"
[133] "Has access" "Has active ing" "Has add monitor ind" "Has admin method" "Has AMP" "Has AMP restr ind" "Has Answer" "Has answer (PPI)" "Has antineopl Rx" "Has antineoplastic" "Has ARP" "Has asso finding"
[145] "Has asso morph" "Has asso proc" "Has basic dose form" "Has basis str subst" "Has biosimilar" "Has brand name" "Has CAP parent item" "Has CAP protocol" "Has CAP value" "Has Category" "Has causative agent" "Has CD category"
[157] "Has chem structure" "Has CI" "Has CI chem class" "Has CI MoA" "Has CI physio effect" "Has clinical course" "Has combi prod ind" "Has comp material" "Has complication" "Has component" "Has conc denom unit" "Has conc denom val"
[169] "Has conc num unit" "Has conc num val" "Has context" "Has count of act ing" "Has count of ing" "Has cytotox chemo Rx" "Has cytotoxic chemo" "Has denomin value" "Has denominator unit" "Has dev intend site" "Has dir device" "Has dir morph"
[181] "Has dir proc site" "Has dir subst" "Has direct site" "Has disc indicator" "Has disp dose form" "Has disposition" "Has dose form" "Has dose form group" "Has dose form unit" "Has drug-drug inter" "Has due to" "Has end date"
[193] "Has endocrine tx" "Has endocrine tx Rx" "Has episodicity" "Has excipient" "Has FDA appr ind" "Has FDA indication" "Has FDA labeling" "Has filling" "Has finding context" "Has finding site" "Has flavor" "Has focus"
[205] "Has form" "Has form continuity" "Has free indicator" "Has Histology ICD0" "Has immunosuppr Rx" "Has immunosuppressor" "Has immunotherapy" "Has immunotherapy Rx" "Has indir device" "Has indir morph" "Has indir proc site" "Has inherent"
[217] "Has inherent loc" "Has intended site" "Has intent" "Has interpretation" "Has interprets" "Has kind" "Has laterality" "Has legal category" "Has licensed route" "Has life circumstan" "Has local therap Rx" "Has local therapy"
[229] "Has manifestation" "Has marketed form" "Has measurement" "Has metabolism" "Has metabolites" "Has method" "Has MoA" "Has modality" "Has modification" "Has non-avail ind" "Has numerator unit" "Has numerator value"
[241] "Has occurrence" "Has off-label ind" "Has ontological form" "Has parent item" "Has part of" "Has pathology" "Has pept-drg cjt Rx" "Has pept-drug cjt" "Has permiss range" "Has physio effect" "Has specimen" "Has specimen morph"
[253] "Has PPI parent code" "Has prec ingredient" "Has precise ing" "Has precondition" "Has precoord pair" "Has priority" "Has proc context" "Has proc device" "Has proc duration" "Has proc morph" "Has spec active ing" "Has supportive med"
[265] "Has prod character" "Has product comp" "Has property" "Has property type" "Has quantified form" "Has question source" "Has radiocjgt Rx" "Has radioconjugate" "Has radiotherapy" "Has radiotherapy Rx" "Has recipient cat" "Has relat context"
[277] "Has relative part" "Has release charact" "Has revision status" "Has role" "Has route" "Has route of admin" "Has scale type" "Has setting" "Has severity" "Has spec active ing" "Has specimen" "Has specimen morph"
[289] "Has specimen proc" "Has specimen source" "Has specimen subst" "Has specimen topo" "Has stage" "Has start date" "Has state of matter" "Has sub-specimen" "Has subject matter" "Has supplier" "Has support med Rx" "Has supportive med"
[301] "Has surface char" "Has surgical appr" "Has system" "Has targeted therapy" "Has targeted tx Rx" "Has technique" "Has temp finding" "Has temporal context" "Has therap class" "Has time aspect" "Has Topography ICD0" "Has trade family grp"
[313] "Has tradename" "Has transformation" "Has type" "Has type of service" "Has unit" "Has unit of admin" "Has unit of presen" "Has unit of prod use" "Has Value" "Has variant" "Has VMP" "Has VRP"
[325] "Histology of ICD0" "Historic treated by" "HOI - MedDRA" "HOI - SNOMED" "ICD0 br to OncoTree" "ICD0 to Chapter" "ICD0 to OncoTree eq" "ICD0 to Proc Schema" "ICD0 to Schema" "Immunosuppressor of" "Immunotherapy of" "Ind/CI - SNOMED"
[337] "Indir device of" "Indir morph of" "Indir proc site of" "Induced by" "Induces" "Is a" "Is off-label ind of" "Inherent location of" "Inheres in" "Inhibits effect" "Intended site of" "Intent of" "Interpretation of"
[349] "Is a" "Is characterized by" "Is CI of" "Is current in" "Is FDA-appr ind of" "Is historical in" "Is off-label ind of" "Is transcribed from" "Is translated from" "Kind of" "Legal category of" "Laterality of"
[361] "Licensed route of" "Life circumstan of" "Local therapy of" "LOINC - CPT4 eq" "Manifestation of" "Mapped from" "Maps to" "Maps to unit" "Maps to value" "Marketed form of" "May be inhibited by" "May be prevented by"
[373] "May be route of" "May be treated by" "May diagnose" "May have route" "May prevent" "Modification of" "MS-DRG - DRG eq" "Multilex has ing" "Measurement of" "MedDRA - HOI" "MedDRA - SMQ" "MedDRA - SNOMED eq"
[385] "Metabolite of" "Method of" "MoA of" "Modality of" "Modification of" "MS-DRG - DRG eq" "Multilex has ing" "Measurement of" "MedDRA - HOI" "MedDRA - SMQ" "MedDRA - SNOMED eq"
[397] "NDFRT has dose form" "NDFRT has ing" "NDFRT ing of" "NDFRT to ATC eq" "NDFRT to VA Class eq" "Nebraska-CAP cat" "Nebraska-CAP eq" "Non-avail ind of" "NDFRT - RxNorm eq" "NDFRT - SNOMED eq" "NDFRT dose form of"
[409] "Occurs before" "OncoTree to ICD0 br" "OncoTree to ICD0 eq" "Ontological form of" "Panel contains" "Paral imprt ind" "Paral imprt of" "Parent item of" "Part of" "Pathology of" "Pept-drug cjt of" "Permiss range of"
[421] "Physiol effect by" "Physiol state of" "PK of" "Plays role" "PPI parent code of" "Prec ingredient of" "Precise ing of" "Precondition of" "Precoord pair of" "Prep to Chem eq" "Priority of" "Proc context of"
[433] "Proc device of" "Proc duration of" "Proc morph of" "Proc Schema to ICD0" "Proc site of" "Process output of" "Prod character of" "Product comp of" "Property of" "Property type of" "Quantified form of" "Question source of"
[445] "Radioconjugate of" "Radiotherapy of" "Recipient cat of" "Relat context of" "Relative part of" "Relative to" "Release charact of" "Revision status of" "Role of" "Role played by" "Route of"
[457] "Route of admin of" "Rx AB-drug cjt of" "Rx antineopl of" "Rx endocrine tx of" "Rx immunosuppr of" "Rx immunotherapy of" "Rx local therapy of" "Rx pept-drg cjt of" "Rx radiocjgt of" "Rx support med of" "Rx targeted tx of"
[469] "RxNorm - ATC" "RxNorm - ATC pr lat" "RxNorm - ATC sec lat" "RxNorm - ATC sec up" "RxNorm - CVX" "RxNorm - DOI" "RxNorm - ETC" "RxNorm - NDFRT eq" "RxNorm - NDFRT name" "RxNorm - SNOMED eq" "RxNorm - Source eq"
[481] "RxNorm - SPL" "RxNorm - VAProd eq" "RxNorm dose form of" "RxNorm has dose form" "RxNorm has ing" "RxNorm inverse is a" "RxNorm is a" "RxNorm - NDFRT eq" "RxNorm - NDFRT name" "RxNorm - SNOMED eq" "RxNorm - Source eq"
[493] "Setting of" "Severity of" "SMQ - MedDRA" "SNOMED - ATC eq" "SNOMED - CPT4 eq" "SNOMED - HOI" "SNOMED - ind/CI" "SNOMED - MedDRA eq" "SNOMED - NDFRT eq" "SNOMED - RxNorm eq" "SNOMED - VA Class eq" "SNOMED cat - CPT4"
[505] "Source - RxNorm eq" "Spec active ing of" "Specimen identity of" "Specimen morph of" "Specimen of" "Specimen proc of" "Specimen subst of" "Specimen topo of" "SPL - RxNorm" "Stage of" "Start date of" "State of matter of"
[517] "Sub-specimen of" "Subject matter of" "Subst used by" "Subsumes" "Supplier of" "Supportive med of" "Surg character of" "Surgical appr of" "System of" "Targeted therapy of" "Technique of" "Temp related to"
[529] "Temporal context of" "Therap class of" "Time aspect of" "Topography of ICD0" "Trade family grp of" "Tradename of" "Transcribes to" "Transformation of" "Translates to" "Type of" "Type of service of" "Unit mapped from"
[541] "Unit of" "Unit of admin of" "Unit of presen of" "Unit of prod use of" "Using acc device" "Using device" "Using energy" "Using finding inform" "Using finding method" "Using subst" "VA Class - SNOMED eq" "VA Class to ATC eq"
[553] "VA Class to NDFRT eq" "Value mapped from" "Value of" "Value to Schema" "VAProd - RxNorm eq" "Variable to Schema" "Variant of" "Version contains" "VMP has prescr stat" "VMP of" "VMP prescr stat of"
[565] "VRP of" "VRP prescr stat of"
```

Concept relationship

In particular, we have the **Maps to** and **Mapped from** relations that can help us to see the mapping between codes.

Example of mapping

```
> cdm$concept_relationship %>% filter(relationship_id == "Maps to") %>% filter(concept_id_1 == 44975210)
# Source:   SQL [1 x 6]
# Database: postgres
  concept_id_1 concept_id_2 relationship_id valid_start_date valid_end_date invalid_reason
      <int>      <int> <chr>           <date>           <date>           <chr>
1    44975210    1790982 Maps to       2009-06-01       2099-12-31       ""
```

Mapping process

Example of mapping

```
> cdm$concept_relationship %>% filter(relationship_id == "Maps to") %>% filter(concept_id_1 == 44975210)
# Source:   SQL [1 x 6]
# Database: postgres
  concept_id_1 concept_id_2 relationship_id valid_start_date valid_end_date invalid_reason
      <int>      <int> <chr>              <date>            <date>            <chr>
1    44975210    1790982 Maps to          2009-06-01        2099-12-31        ""
> cdm$concept %>% filter(concept_id %in% c(44975210, 1790982))
# Source:   SQL [2 x 10]
# Database: postgres
  concept_id concept_name      domain_id vocabulary_id concept_class_id standard_concept concept_code valid_start_date valid_end_date
      <int> <chr>                <chr>      <chr>        <chr>          <chr>          <chr>      <date>        <date>
1    1790982 chlorhexidine gluconate 1.2 MG/ML Mouthwash Drug      RxNorm      Clinical Drug      S          834127      2009-03-01      2099-12-31
2    44975210 chlorhexidine gluconate 1.2 MG/ML Mouthwash Drug      NDC         11-digit NDC      NA          00116200116 2009-06-01      2099-12-31
```

Mapping process

Example of mapping

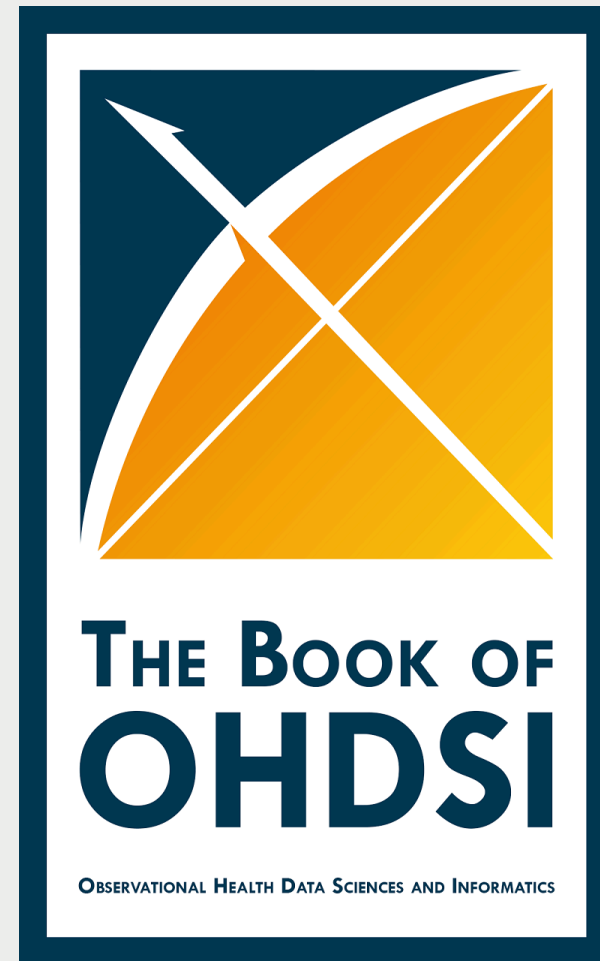
```
> cdm$concept_relationship %>% filter(relationship_id == "Maps to") %>% filter(concept_id_1 == 44975210)
# Source:   SQL [1 x 6]
# Database: postgres
  concept_id_1 concept_id_2 relationship_id valid_start_date valid_end_date invalid_reason
      <int>      <int> <chr>              <date>           <date>           <chr>
1    44975210    1790982 Maps to          2009-06-01      2099-12-31      ""
> cdm$concept %>% filter(concept_id %in% c(44975210, 1790982))
# Source:   SQL [2 x 10]
# Database: postgres
  concept_id concept_name          domain_id vocabulary_id concept_class_id standard_concept concept_code valid_start_date valid_end_date
      <int> <chr>                  <chr>      <chr>          <chr>          <chr>          <chr>      <date>           <date>
1    1790982 chlorhexidine gluconate 1.2 MG/ML Mouthwash Drug      RxNorm        Clinical Drug      S            834127      2009-03-01      2099-12-31
2    44975210 chlorhexidine gluconate 1.2 MG/ML Mouthwash Drug      NDC           11-digit NDC      NA            00116200116 2009-06-01      2099-12-31
> cdm$drug_exposure %>%
+   filter(drug_concept_id == 1790982) %>%
+   select(drug_exposure_id, drug_concept_id, drug_exposure_start_date, drug_exposure_end_date, drug_source_concept_id)
# Source:   SQL [?? x 5]
# Database: postgres
  drug_exposure_id drug_concept_id drug_exposure_start_date drug_exposure_end_date drug_source_concept_id
      <int64>      <int> <date>              <date>              <int>
1    123429592    1790982 2017-12-15          2017-12-30          44975210
2    778541702    1790982 2018-02-08          2018-02-23          44975210
3    4043828552    1790982 2018-01-05          2018-01-20          44975210
4    2594409161    1790982 2018-04-24          2018-05-25          44975210
5    1942091989    1790982 2017-01-24          2017-02-08          44975210
6    1857857471    1790982 2021-11-10          2021-11-25          44975210
7    2801754279    1790982 2021-11-10          2021-11-25          44975210
8    3908339319    1790982 2017-04-05          2017-04-18          44975210
9    2123508505    1790982 2017-04-12          2017-04-25          44975210
10   1310852067    1790982 2017-03-15          2017-03-28          44975210
# i more rows
# i Use `print(n = ...)` to see more rows
>
```

Mapping process

More details

For more details on how the vocabularies work you can check: **Vocabulary course in EHDEN academy**

All details about OMOP CDM and more can be found in: **the book of ohdsi.**



The book of ohdsi cover

Let's start coding

These are the packages that we will use in this presentation:

```
library(omock)
library(DBI)
library(CDMConnector)
library(duckdb)
library(here)
library(dplyr)
library(omopgenerics)
library(visOmopResults)
```

You can click on the specific functions to see **?help** and what package they come from:

```
cdmFromCon()
```

Create a mock reference

You can create a mock database using *omock* from one of the **28** available synthetic databases:

```
availableMockDatasets()
```

```
[1] "delphi-100k"                "delphi-100k_5.4"          "empty_cdm"
[4] "empty_cdm_5.3"             "empty_cdm_5.4"           "GiBleed"
[7] "synpuf-1k"                 "synpuf-1k_5.3"           "synpuf-1k_5.4"
[10] "synthea-allergies-10k"     "synthea-breast_cancer-10k" "synthea-contraceptives-10k"
[13] "synthea-covid19-10k"       "synthea-covid19-200k"     "synthea-dermatitis-10k"
[16] "synthea-heart-10k"         "synthea-hiv-10k"          "synthea-lung_cancer-10k"
[19] "synthea-medications-10k"   "synthea-metabolic_syndrome-10k" "synthea-opioid_addiction-10k"
[22] "synthea-rheumatoid_arthritis-10k" "synthea-snf-10k"         "synthea-surgery-10k"
[25] "synthea-total_joint_replacement-10k" "synthea-veteran_prostate_cancer-10k" "synthea-veterans-10k"
[28] "synthea-weight_loss-10k"
```

```
cdm <- mockCdmFromDataset(datasetName = "GiBleed")
```

```
i Loading bundled GiBleed tables from package data.
i Adding drug_strength table.
i Creating local <cdm_reference> object.
```

```
cdm
```

— # OMOP CDM reference (local) of GiBleed —

- omop tables: care_site, cdm_source, concept, concept_ancestor, concept_class, concept_relationship, concept_synonym, condition_era, condition_occurrence, cost, death, device_exposure, domain, dose_era, drug_era, drug_exposure, drug_strength, fact_relationship, location, measurement, metadata, note, note_nlp, observation, observation_period, payer_plan_period, person, procedure_occurrence, provider, relationship, source_to_concept_map, specimen, visit_detail, visit_occurrence, vocabulary

- cohort tables: -
- achilles tables: -
- other tables: -

<cdm_reference> object

```
class(cdm)
```

```
[1] "cdm_reference"
```

```
cdmName(cdm)
```

```
[1] "GiBleed"
```

```
cdmVersion(cdm)
```

```
[1] "5.3"
```

```
cdmSource(cdm)
```

This is a local cdm source

<cdm_table> object

```
cdm$person
```

```
# A tibble: 2,694 × 18
  person_id gender_concept_id year_of_birth month_of_birth day_of_birth birth_datetime race_concept_id
*   <int>         <int>         <int>         <int>         <int> <dtm>         <int>
1         6         8532         1963          12          31 1963-12-31 00:00:00      8516
2        123         8507         1950           4          12 1950-04-12 00:00:00      8527
3        129         8507         1974          10           7 1974-10-07 00:00:00      8527
4         16         8532         1971          10          13 1971-10-13 00:00:00      8527
5         65         8532         1967           3          31 1967-03-31 00:00:00      8516
6         74         8532         1972           1           5 1972-01-05 00:00:00      8527
7         42         8532         1909          11           2 1909-11-02 00:00:00      8527
8        187         8507         1945           7          23 1945-07-23 00:00:00      8527
9         18         8532         1965          11          17 1965-11-17 00:00:00      8527
10        111         8532         1975           5           2 1975-05-02 00:00:00      8527
# i 2,684 more rows
```

```
class(cdm$person)
```

```
[1] "omop_table" "cdm_table"  "tbl_df"     "tbl"        "data.frame"
```

Connecting to a database from R (the DBI package)

In general the *OMOP* datasets that we will use won't be locally, and they will on a database.

Database connections from R can be made using the **DBI** package.

Connect to postgres:

```
library(RPostgres)
db <- dbConnect(
  Postgres(),
  dbname = "...",
  host = "...",
  user = "...",
  password = "..."
)
```

Connecting to a database from R (the DBI package)

Connect to Sql server:

```
library(odbc)
db <- dbConnect(
  odbc(),
  Driver      = "ODBC Driver 18 for SQL Server",
  Server      = "...",
  Database    = "...",
  UID         = "...",
  PWD         = "...",
  TrustServerCertificate = "yes",
  Port        = "..."
)
```

In this [CDMConnector article](#) you can see how to connect to the different supported DBMS.

Databases organisation

Databases are organised by **schemas** (blueprint or plan that defines how the data will be organised and structured within the database).

In general, OMOP databases have two schemas:

- **cdm schema**: it contains all the tables of the cdm. Usually we only will have reading permission for this schema.
- **write schema**: it is a place where we can store tables (like cohorts). We need writing permissions to this schema.

Connect to duckdb database

duckdb is a package that allows us to work with local databases.

You should see that in the *Database* folder there are three datasets:

- **GiBleed**
- **Delphi**
- **Synpuf**

Let's create our first cdm reference

```
# connect to the database
dbdir <- here("Databases", "GiBleed.duckdb")
con <- dbConnect(drv = duckdb(dbdir = dbdir))
cdm <- cdmFromCon(con = con, cdmSchema = "main", writeSchema = "results")
```

Note: method with signature 'DBIConnection#Id' chosen for function 'dbExistsTable',
target signature 'duckdb_connection#Id'.
"duckdb_connection#ANY" would also be valid

```
cdm
```

— # OMOP CDM reference (duckdb) of Synthea —

- omop tables: care_site, cdm_source, concept, concept_ancestor, concept_class, concept_relationship, concept_synonym, condition_era, condition_occurrence, cost, death, device_exposure, domain, dose_era, drug_era, drug_exposure, drug_strength, fact_relationship, location, measurement, metadata, note, note_nlp, observation, observation_period, payer_plan_period, person, procedure_occurrence, provider, relationship, source_to_concept_map, specimen, visit_detail, visit_occurrence, vocabulary
- cohort tables: -
- achilles tables: -
- other tables: -

Access to tables of the cdm reference

```
cdm$person
```

```
# A query:  ?? x 18
# Database: DuckDB 1.5.4 [unknown@Linux 6.17.0-1018-azure:R
4.6.1//home/runner/work/RealWorldEvidenceSummerSchool2026/RealWorldEvidenceSummerSchool2026/Databases/GiBleed.duckdb]
  person_id gender_concept_id year_of_birth month_of_birth day_of_birth birth_datetime      race_concept_id
      <int>         <int>         <int>         <int>         <int> <dtm>              <int>
1          6          8532          1963           12           31 1963-12-31 00:00:00      8516
2         123          8507          1950            4           12 1950-04-12 00:00:00      8527
3         129          8507          1974           10            7 1974-10-07 00:00:00      8527
4          16          8532          1971           10           13 1971-10-13 00:00:00      8527
5          65          8532          1967            3           31 1967-03-31 00:00:00      8516
6          74          8532          1972            1            5 1972-01-05 00:00:00      8527
7          42          8532          1909           11            2 1909-11-02 00:00:00      8527
8         187          8507          1945            7           23 1945-07-23 00:00:00      8527
9          18          8532          1965           11           17 1965-11-17 00:00:00      8527
--          --          --          --          --          --          --
```

Read tables in GiBleed

Once we read a table we can operate with it and for example count the number of rows of person table.

```
cdm$person |>  
  count()
```

```
# A query:  ?? x 1
```

```
# Database: DuckDB 1.5.4 [unknown@Linux 6.17.0-1018-azure:R
```

```
4.6.1//home/runner/work/RealWorldEvidenceSummerSchool2026/RealWorldEvidenceSummerSchool2026/Databases/GiBleed.duckdb]
```

```
      n
```

```
<dbl>
```

```
1    2694
```

Operation with tidyverse

If you are familiarised with **tidyverse** you can use any of the usual **dplyr** commands in you database tables.

```
cdm$drug_exposure |>
  group_by(drug_concept_id) |>
  summarise(number_persons = n_distinct(person_id)) |>
  collect() |>
  arrange(desc(number_persons))
```

```
# A tibble: 113 × 2
  drug_concept_id number_persons
      <int>         <dbl>
1      40213227         2660
2       1127433         2580
3      40213160         2140
4       1713671         2021
5      19059056         1927
6       1118084         1844
7      40213296         1737
8      40213306         1560
9       1127078         1428
10     40229134         1393
# i 103 more rows
```

Database name

When we have a cdm object we can check which is the name of that database using:

```
cdmName(cdm)
```

```
[1] "Synthea"
```

In some cases we want to give a database a name that we want, this can be done at the connection stage:

```
cdm <- cdmFromCon(  
  con = con, cdmSchema = "main", writeSchema = "results", cdmName = "GiBleed"  
)
```

```
cdmName(cdm)
```

```
[1] "GiBleed"
```

Create a new table

Let's say I want to subset the **condition_occurrence** table to a certain rows and certain columns and save it so I can later access it.

temporary table (default):

```
cdm$my_saved_table <- cdm$condition_occurrence |>
  filter(condition_concept_id == 4112343) |>
  select(person_id, condition_start_date) |>
  compute()
listSourceTables(cdm)
```

character(0)

Create a new table

permanent table:

```
cdm$my_saved_table <- cdm$condition_occurrence |>  
  filter(condition_concept_id == 4112343) |>  
  select(person_id, condition_start_date) |>  
  compute(name = "my_saved_table")  
listSourceTables(cdm)
```

```
[1] "my_saved_table"
```

Create a new table

```
cdm
```

```
— # OMOP CDM reference (duckdb) of GiBleed
```

- omop tables: care_site, cdm_source, concept, concept_ancestor, concept_class, concept_relationship, concept_synonym, condition_era, condition_occurrence, cost, death, device_exposure, domain, dose_era, drug_era, drug_exposure, drug_strength, fact_relationship, location, measurement, metadata, note, note_nlp, observation, observation_period, payer_plan_period, person, procedure_occurrence, provider, relationship, source_to_concept_map, specimen, visit_detail, visit_occurrence, vocabulary
- cohort tables: -
- achilles tables: -
- other tables: my_saved_table

```
cdm$my_saved_table
```

```
# A query: ?? x 2
```

```
# Database: DuckDB 1.5.4 [unknown@Linux 6.17.0-1018-azure:R
```

```
4.6.1//home/runner/work/RealWorldEvidenceSummerSchool2026/RealWorldEvidenceSummerSchool2026/Databases/GiBleed.duckdb]
```

```
  person_id condition_start_date
```

```
    <int> <date>
```

```
1      263 2015-10-02
2      439 1990-03-20
3      449 1999-12-12
4      515 1961-11-14
5       17 1963-12-02
6       30 1993-03-19
7       90 1970-01-15
8      116 1959-06-11
9      137 2005-11-15
```


Drop an existing table

To drop an existing table:

- Eliminate the table from the cdm object.
- Eliminate the table from the database.

```
cdm <- dropSourceTable(cdm = cdm, name = "my_saved_table")  
cdm
```

— # OMOP CDM reference (duckdb) of GiBleed —

- omop tables: care_site, cdm_source, concept, concept_ancestor, concept_class, concept_relationship, concept_synonym, condition_era, condition_occurrence, cost, death, device_exposure, domain, dose_era, drug_era, drug_exposure, drug_strength, fact_relationship, location, measurement, metadata, note, note_nlp, observation, observation_period, payer_plan_period, person, procedure_occurrence, provider, relationship, source_to_concept_map, specimen, visit_detail, visit_occurrence, vocabulary
- cohort tables: -
- achilles tables: -
- other tables: -

Drop an existing table

```
listSourceTables(cdm)
```

```
character(0)
```

Drop an existing table

Let's drop also the other table that we created:

```
cdm <- dropSourceTable(cdm = cdm, name = starts_with("og_"))
cdm
```

— # OMOP CDM reference (duckdb) of GiBleed —

- omop tables: care_site, cdm_source, concept, concept_ancestor, concept_class, concept_relationship, concept_synonym, condition_era, condition_occurrence, cost, death, device_exposure, domain, dose_era, drug_era, drug_exposure, drug_strength, fact_relationship, location, measurement, metadata, note, note_nlp, observation, observation_period, payer_plan_period, person, procedure_occurrence, provider, relationship, source_to_concept_map, specimen, visit_detail, visit_occurrence, vocabulary
- cohort tables: -
- achilles tables: -
- other tables: -

Drop an existing table

```
listSourceTables(cdm)
```

```
character(0)
```

Insert a table

Let's say we have a local tibble and we want to insert it in the cdm:

```
cdm <- insertTable(cdm = cdm, name = "my_test_table", table = cars)
cdm
```

— # OMOP CDM reference (duckdb) of GiBleed —

- omop tables: care_site, cdm_source, concept, concept_ancestor, concept_class, concept_relationship, concept_synonym, condition_era, condition_occurrence, cost, death, device_exposure, domain, dose_era, drug_era, drug_exposure, drug_strength, fact_relationship, location, measurement, metadata, note, note_nlp, observation, observation_period, payer_plan_period, person, procedure_occurrence, provider, relationship, source_to_concept_map, specimen, visit_detail, visit_occurrence, vocabulary
- cohort tables: -
- achilles tables: -
- other tables: my_test_table

Insert a table

```
listSourceTables(cdm)
```

```
[1] "my_test_table"
```

```
cdm$my_test_table
```

```
# A query: ?? x 2
```

```
# Database: DuckDB 1.5.4 [unknown@Linux 6.17.0-1018-azure:R
```

```
4.6.1//home/runner/work/RealWorldEvidenceSummerSchool2026/RealWorldEvidenceSummerSchool2026/Databases/GiBleed.duckdb]
```

	speed	dist
	<dbl>	<dbl>
1	4	2
2	4	10
3	7	4
4	7	22
5	8	16
6	9	10
7	10	18
8	10	26
9	10	34

Use a prefix

It is **VERY IMPORTANT** that when we create the cdm object we use a prefix:

```
cdm <- cdmFromCon(  
  con = con,  
  cdmSchema = "main",  
  writeSchema = "results",  
  writePrefix = "my_prefix_"  
)  
cdm
```

— # OMOP CDM reference (duckdb) of Synthea —

- omop tables: care_site, cdm_source, concept, concept_ancestor, concept_class, concept_relationship, concept_synonym, condition_era, condition_occurrence, cost, death, device_exposure, domain, dose_era, drug_era, drug_exposure, drug_strength, fact_relationship, location, measurement, metadata, note, note_nlp, observation, observation_period, payer_plan_period, person, procedure_occurrence, provider, relationship, source_to_concept_map, specimen, visit_detail, visit_occurrence, vocabulary
- cohort tables: -
- achilles tables: -
- other tables: -

Use a prefix

Now when we create a new table the prefix will be automatically added:

```
cdm <- insertTable(cdm = cdm, name = "my_test_table", table = cars)
cdm
```

— # OMOP CDM reference (duckdb) of Synthea —

- omop tables: care_site, cdm_source, concept, concept_ancestor, concept_class, concept_relationship, concept_synonym, condition_era, condition_occurrence, cost, death, device_exposure, domain, dose_era, drug_era, drug_exposure, drug_strength, fact_relationship, location, measurement, metadata, note, note_nlp, observation, observation_period, payer_plan_period, person, procedure_occurrence, provider, relationship, source_to_concept_map, specimen, visit_detail, visit_occurrence, vocabulary
- cohort tables: -
- achilles tables: -
- other tables: my_test_table

Use a prefix

```
listSourceTables(cdm = cdm)
```

```
[1] "my_test_table"
```

```
cdm$my_test_table
```

```
# A query: ?? x 2
```

```
# Database: DuckDB 1.5.4 [unknown@Linux 6.17.0-1018-azure:R
```

```
4.6.1//home/runner/work/RealWorldEvidenceSummerSchool2026/RealWorldEvidenceSummerSchool2026/Databases/GiBleed.duckdb]
```

	speed	dist
	<dbl>	<dbl>
1	4	2
2	4	10
3	7	4
4	7	22
5	8	16
6	9	10
7	10	18
8	10	26
9	10	34

Use a prefix

DO NOT use the prefix to drop tables, you only care about the prefix at the connection stage!

```
cdm <- dropSourceTable(cdm = cdm, name = "my_prefix_my_test_table")
listSourceTables(cdm = cdm)

[1] "my_test_table"
```

Use a prefix

cdm

— # OMOP CDM reference (duckdb) of Synthea —

- omop tables: care_site, cdm_source, concept, concept_ancestor, concept_class, concept_relationship, concept_synonym, condition_era, condition_occurrence, cost, death, device_exposure, domain, dose_era, drug_era, drug_exposure, drug_strength, fact_relationship, location, measurement, metadata, note, note_nlp, observation, observation_period, payer_plan_period, person, procedure_occurrence, provider, relationship, source_to_concept_map, specimen, visit_detail, visit_occurrence, vocabulary
- cohort tables: -
- achilles tables: -
- other tables: my_test_table

Use a prefix

DO NOT use the prefix to drop tables, you only care about the prefix at the connection stage!

```
cdm <- dropSourceTable(cdm = cdm, name = "my_test_table")  
listSourceTables(cdm = cdm)  
character(0)
```

Use a prefix

cdm

— # OMOP CDM reference (duckdb) of Synthea

- omop tables: care_site, cdm_source, concept, concept_ancestor, concept_class, concept_relationship, concept_synonym, condition_era, condition_occurrence, cost, death, device_exposure, domain, dose_era, drug_era, drug_exposure, drug_strength, fact_relationship, location, measurement, metadata, note, note_nlp, observation, observation_period, payer_plan_period, person, procedure_occurrence, provider, relationship, source_to_concept_map, specimen, visit_detail, visit_occurrence, vocabulary
- cohort tables: -
- achilles tables: -
- other tables: -

Consistency rules

We use `compute()` to compute the result into a temporary (`temporary = TRUE`) or permanent (`temporary = FALSE`) table.

If it is a temporary table we can assign it to where I want for example:

```
cdm$my_custom_name <- cdm$person |>  
  compute()
```

If it is a permanent table we can only assign it to the same name:

error:

```
cdm$my_custom_name <- cdm$person |>  
  compute(name = "not_my_custom_name")
```

Error in `[<-`:

✖ You can't assign a table named `not_my_custom_name` to `my_custom_name`.

i You can change the name using `compute`:

```
cdm[['my_custom_name']] <- yourObject |>  
  dplyr::compute(name = 'my_custom_name')
```

i You can also change the name using the ``name`` argument in your function: ``name` = 'my_custom_name'`.

no error:

```
cdm$my_custom_name <- cdm$person |>  
  compute(name = "my_custom_name")
```


Consistency rules

Omop names are reserved words, e.g. we can not assign a table that is not the person table to **cdm\$person**.

```
cdm$person <- cdm$drug_exposure |>  
  compute(name = "person", temporary = FALSE)
```

```
Error in `newOmopTable()`:  
! gender_concept_id, year_of_birth, race_concept_id and ethnicity_concept_id are not present in table person
```

```
cdm$drug_exposure <- cdm$drug_exposure |>  
  rename("my_id" = "person_id") |>  
  compute(name = "drug_exposure", temporary = FALSE)
```

```
Error in `newOmopTable()`:  
! person_id is not present in table drug_exposure
```

Result model

The output of our analyses has been standardised to the `<summarised_result>` object.

```
x <- mockSummarisedResult()
x

# A tibble: 126 × 13
  result_id cdm_name group_name group_level strata_name      strata_level variable_name variable_level estimate_name
    <int> <chr>      <chr>      <chr>      <chr>      <chr>      <chr>      <chr>      <chr>
1         1 mock      cohort_name cohort1      overall      overall      number subje... <NA>      count
2         1 mock      cohort_name cohort1      age_group &&& sex <40 &&& Male number subje... <NA>      count
3         1 mock      cohort_name cohort1      age_group &&& sex >=40 &&& Male number subje... <NA>      count
4         1 mock      cohort_name cohort1      age_group &&& sex <40 &&& Fema... number subje... <NA>      count
5         1 mock      cohort_name cohort1      age_group &&& sex >=40 &&& Fem... number subje... <NA>      count
6         1 mock      cohort_name cohort1      sex          Male          number subje... <NA>      count
7         1 mock      cohort_name cohort1      sex          Female         number subje... <NA>      count
8         1 mock      cohort_name cohort1      age_group    <40          number subje... <NA>      count
9         1 mock      cohort_name cohort1      age_group    >=40         number subje... <NA>      count
10        1 mock      cohort_name cohort2      overall      overall      number subje... <NA>      count
# i 116 more rows

class(x)

[1] "summarised_result" "omop_result"      "tbl_df"           "tbl"              "data.frame"
```


<summarised_result>

The summarised result object contains 13 columns:

```
glimpse(x)
Rows: 126
Columns: 13
$ result_id      <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...
$ cdm_name       <chr> "mock", "mock", "mock", "mock", "mock", "mock", "mock", "mock", "mock", "mock", "mock", "mock", "mock...
$ group_name     <chr> "cohort_name", "cohort_name", "cohort_name", "cohort_name", "cohort_name", "cohort_name", "coho...
$ group_level    <chr> "cohort1", "cohort1", "cohort1", "cohort1", "cohort1", "cohort1", "cohort1", "cohort1", "coho...
$ strata_name    <chr> "overall", "age_group &&& sex", "age_group &&& sex", "age_group &&& sex", "age_group &&& sex"...
$ strata_level   <chr> "overall", "<40 &&& Male", ">=40 &&& Male", "<40 &&& Female", ">=40 &&& Female", "Male", "Fem...
$ variable_name  <chr> "number subjects", "number subjects", "number subjects", "number subjects", "number subjects"...
$ variable_level <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N...
$ estimate_name  <chr> "count", "count", "count", "count", "count", "count", "count", "count", "count", "count", "co...
$ estimate_type  <chr> "integer", "integer", "integer", "integer", "integer", "integer", "integer", "integer", "inte...
$ estimate_value <chr> "2655087", "3721239", "5728534", "9082078", "2016819", "8983897", "9446753", "6607978", "6291...
$ additional_name <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "over...
```

<summarised_result>

And have some associated settings

```
settings(x)

# A tibble: 1 × 8
  result_id result_type package_name package_version group strata additional min_cell_count
  <int> <chr> <chr> <chr> <chr> <chr> <chr> <chr>
1      1 mock_summarised_result vis0mopResults 1.5.0 cohort_name age_group &&& s... "" 0
```

tidy the result object

```
x |>  
  tidy() |>  
  glimpse()
```

```
Rows: 72  
Columns: 10  
$ cdm_name      <chr> "mock", "mock", "mock", "mock", "mock", "mock", "mock", "mock", "mock", "mock", "mock", "mock", ...  
$ cohort_name   <chr> "cohort1", "cohort1", "cohort1", "cohort1", "cohort1", "cohort1", "cohort1", "cohort1", "cohort1", "cohort1", "cohort1", "cohort1", ...  
$ age_group     <chr> "overall", "<40", ">=40", "<40", ">=40", "overall", "overall", "<40", ">=40", "overall", "<40", ...  
$ sex           <chr> "overall", "Male", "Male", "Female", "Female", "Male", "Female", "overall", "overall", "overall", ...  
$ variable_name <chr> "number subjects", "number subjects", "number subjects", "number subjects", "number subjects", ...  
$ variable_level <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...  
$ count         <int> 2655087, 3721239, 5728534, 9082078, 2016819, 8983897, 9446753, 6607978, 6291140, 617863, 205974, ...  
$ mean          <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 38.003518, 77.744522, 9...  
$ sd            <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 7.942399, 1.079436, 7.2...  
$ percentage     <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
```

Other objects and classes

- `<odelist>`, `<odelist_with_details>`, `<conceptSetExpression>`
- `<cohort_table>`
- `<achilles_table>`

omock

👉 Packages website

👉 CRAN link

👉 Manual

✉️ mike.du@ndorms.ox.ac.uk



omopgenerics

👉 Packages website

👉 CRAN link

👉 Manual

✉️ marti.catalasabate@ndorms.ox.ac.uk

visOmopResults

👉 Packages website

👉 CRAN link

👉 Manual



nuria.mercadebesora@ndorms.ox.ac.uk



CDMConnector

👉 Packages website

👉 CRAN link

👉 Manual

✉️ marti.catalasabate@ndorms.ox.ac.uk

